Building One Coherent System

Phase 1–3 Review & Stage 4 Readiness

Gemini Notebook

CONTINUOUS ASSESSMENT		
Assessment instrument	What to submit	Weight
Project Deliverable 1	PROJECT DOCUMENTATION: • PROJECT PROPOSAL + SYSTEM REQUIREMENT SPECIFICATION + SYSTEM DESIGN SPECIFICATION	40
Project Deliverable 2	PROJECT IMPLEMENTATION	50
Project Deliverable 3	PHASE 5 - PRESENTATION	10

Project identity is stable, but engineering alignment is weak.

THE EVIDENCE

45 Phase 1 Projects

48 Phase 2 Reports

50 Phase 3 Reports

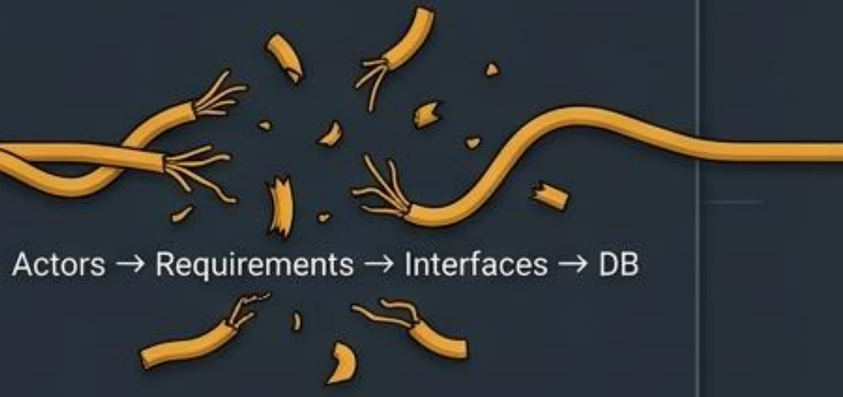
PHASE 2 READINESS



Approximately two-thirds of Phase 2 submissions require more than minor formatting changes.

THE VERDICT

Most students are working on the same general project across phases, but the detailed engineering alignment (Actors → Requirements → Interfaces → DB) is fractured.



Actors → Requirements → Interfaces → DB

Gemini Notebook

The Root Cause: Treating phases as separate assignments instead of living documentation.

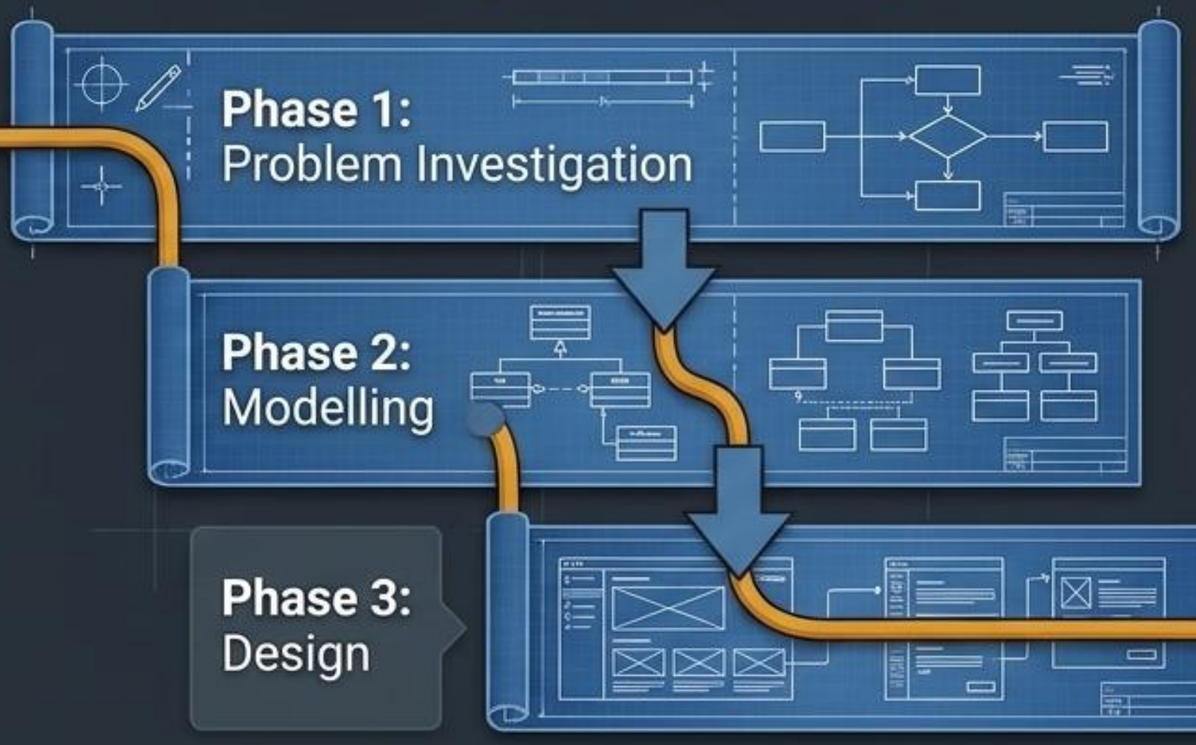
The Illusion

Phase 1: A report about a problem

Phase 2: A report with UML diagrams

Phase 3: A collection of screens

The Engineering Reality



These were not intended to be three unrelated documents. They must collectively answer one question:
What system are you building, and **how will we know the implementation is correct?**

Gemini Notebook

Building One Coherent System

Class Feedback and Diagnostics on Phases 1–3

The Intent

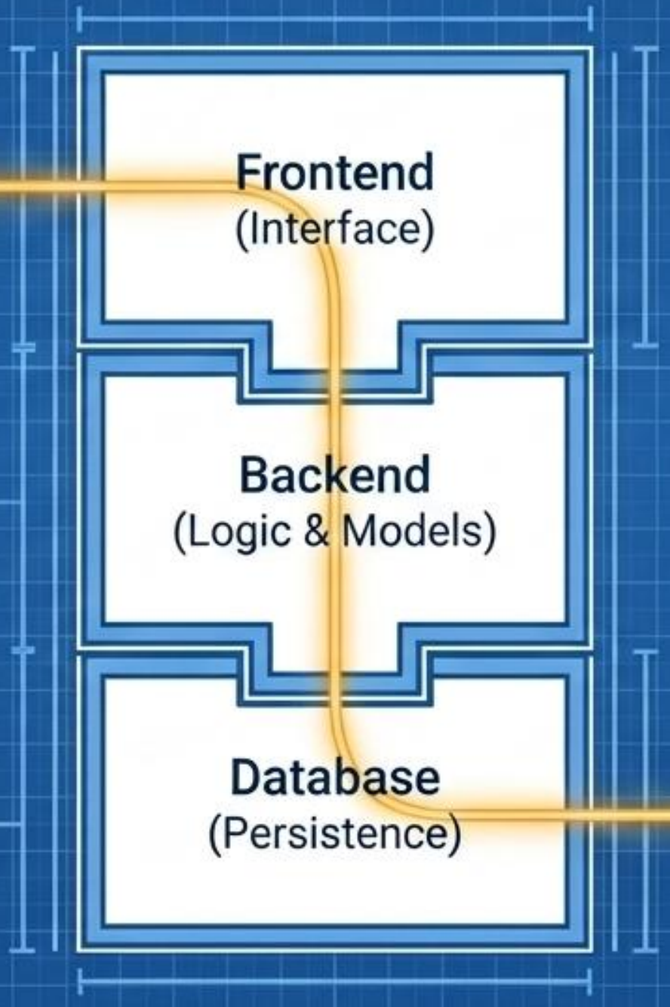
Evaluate whether Phases 1–3 provide a reliable, engineered foundation for Stage 4 implementation.

The Reality

Found promising ideas, but weak engineering alignment.
Two-thirds of submissions require targeted or substantial correction.

The Goal

Establish a traceable, implementation-ready baseline before writing a single line of code.



Gemini Notebook

The Principal Problem is Disconnected Documentation

The Cohort Pattern: Isolated Assignments



Symptoms:

- Phase 2 actors not in Phase 1
- Phase 3 screens with features missing from Phase 2
- Changing system names and shifting terminology

The Expectation: A Living Document



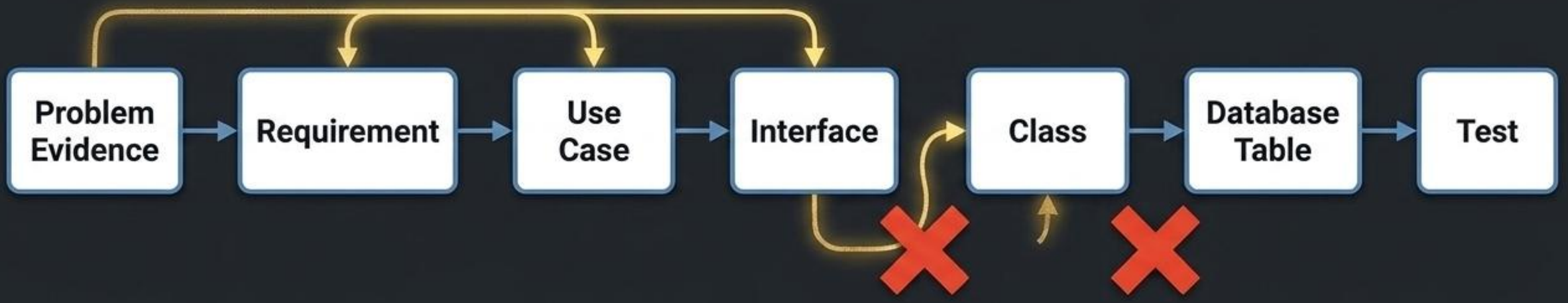
The Standard:
Progressively refining a single, coherent system where every change is documented, justified, and carried forward.

The Stage 4 Risk: **When implementation begins, you will not know which document controls the project—resulting in a final system that no longer solves the original problem.**

Gemini Notebook

Formal Traceability is the Missing Golden Thread

Traceability Chain



The Finding

Only ~2 reports used searchable traceability terminology. Identifiers (e.g., FR-01) vanished entirely by Phase 3.

The Mandate

You must be able to trace a requirement from its source through design, implementation, and testing.

Self-Check

Can you select your five most important requirements and pinpoint exactly which database table and backend component implements them?

Gemini Notebook

Ungrounded Investigation Leads to Uncontrolled Scope

Phase 1: Investigation

Generic Interview → **Vague Feature List** →  **Scope Creep** (National Hospital/ City Platform) → **Unfinished Pages**

The Evidence Gap:
Missing interview dates, unspecified clients, and generic daily activity summaries.

The Consequence:
Without grounded evidence, systems mutated from manageable departmental applications into massive, unbuildable networks.

Specific Observed Transaction → **Targeted Requirement** → **Minimum Viable Product (MVP)** → **Tested Core Workflow**

The Engineering Standard: Define your MVP and explicit exclusions. A smaller, complete, and tested system is professionally valuable. A massive collection of disconnected screens is not.

Gemini Notebook

Engineering Precision: Constraining the Solution

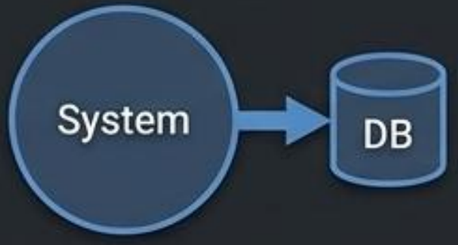
	Superficial Feature (Red Flag)	Engineered Requirement (Green Light)
Functional Requirements	The system must allow booking.	**FR-07**: An authenticated student must be able to request an available resource for a specified date and time. System checks for overlapping approved bookings.
Non-Functional Requirements (NFRs)	The system should respond quickly.	**NFR-04**: In the deployed university test environment, 95% of API requests must complete within 3 seconds with 30 active users.

Why Precision Matters for Stage 4: Vague requirements allow AI tools to invent booking rules, user roles, and database tables. Precision constrains the generated solution and defines your exact acceptance criteria.

Gemini Notebook

Models Must Guide Implementation, Not Just Earn Marks

Context Diagrams



System boundaries blurred; internal databases incorrectly shown as external entities.

Activity Diagrams



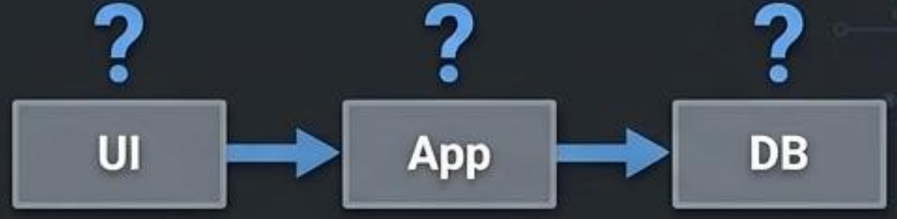
Only the successful 'happy path' shown; no decision conditions, alternate flows, or error handling.

State Diagrams



GUI pages modeled as states (Login/Logout) rather than modeling true entity lifecycles (Booking, Incident, Payment).

Architecture Diagrams



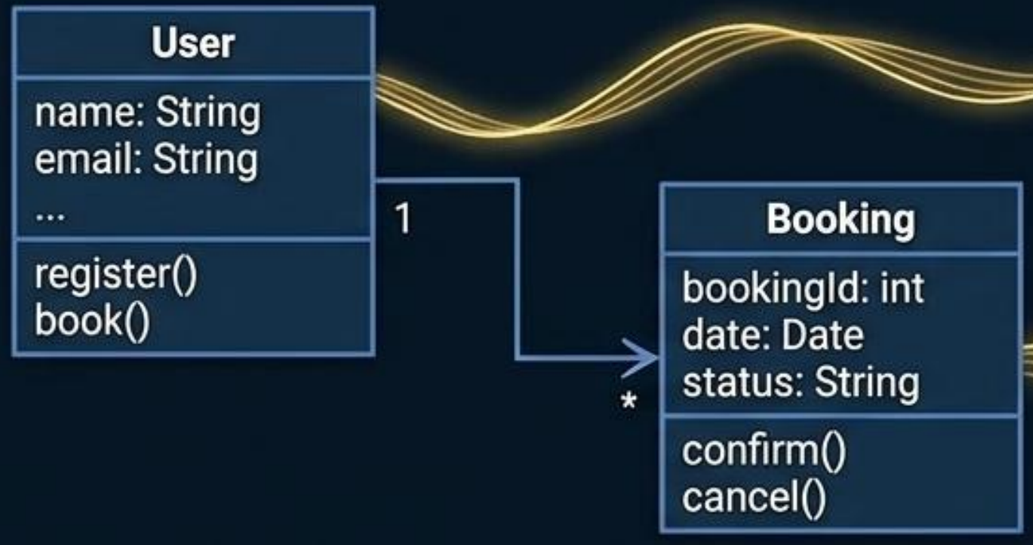
Generic "3-box" diagrams totally disconnected from actual proposed tech stacks (React, Firebase, PostgreSQL).

The Rule: A diagram that conflicts with your database or UI is worse than no diagram at all. If it does not guide code structure, it must be corrected.

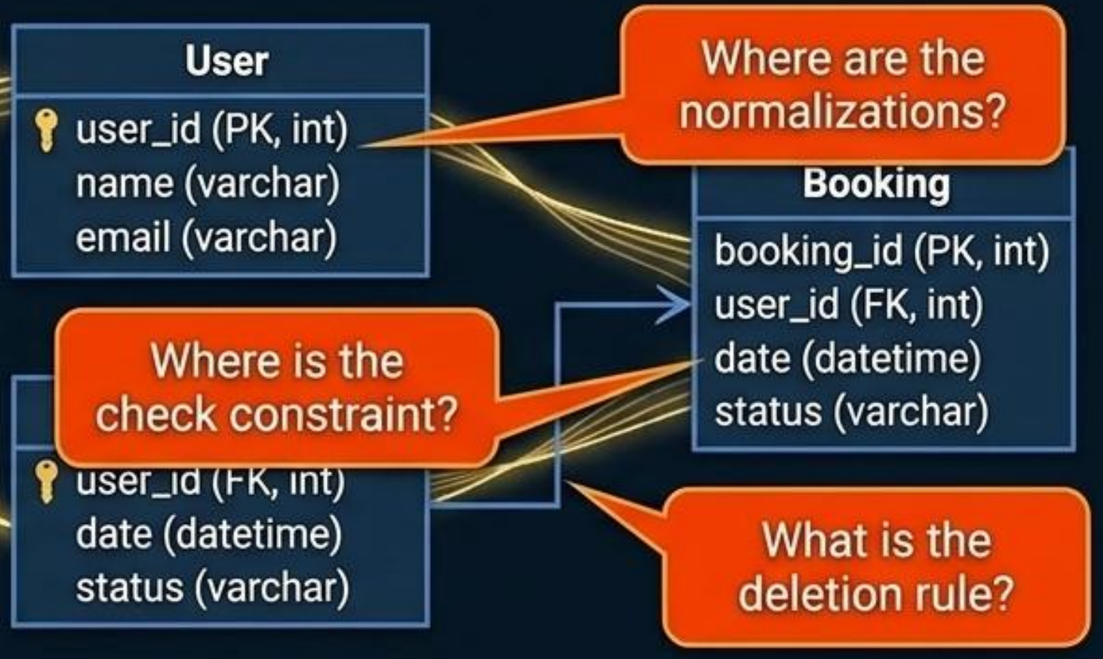
Gemini Notebook

Reconciling the Domain Model with Database Reality

Conceptual Domain (Phase 2)



Database Reality (Phase 3)



The Finding

Domain classes disappeared; tables lacked constraints; GUI form fields were mapped directly into database columns.

The Risk

Without engineering principles (normalization, indexing, cardinality), AI-generated plausible-looking schemas will allow duplicate records, double bookings, and orphan records.

Gemini Notebook

The Illusion of the “Happy Path” Interface



What was submitted (Appearance)

Waterline

Massive & Hidden Reality

- Empty Data State
- Loading / Timeouts
- Offline Mode
- Invalid User Input
- Unauthorized Access
- Conflicting Updates
- Rolled-back Transactions

The Finding

Most reports concentratstrictly on the successful user journey.

Only ~5 reports contained evidence of empty or error states.

The Correction

Real systems must handle failed requests, timeouts, and missing fields.
You must design the system’s behavior when things go wrong.

Create a comprehensive screen inventory.

Gemini Notebook

Transactional Integrity and Concurrency



The Core Principle

Systems involve transactions where several things must happen together, or not happen at all. Interfaces alone cannot enforce this.

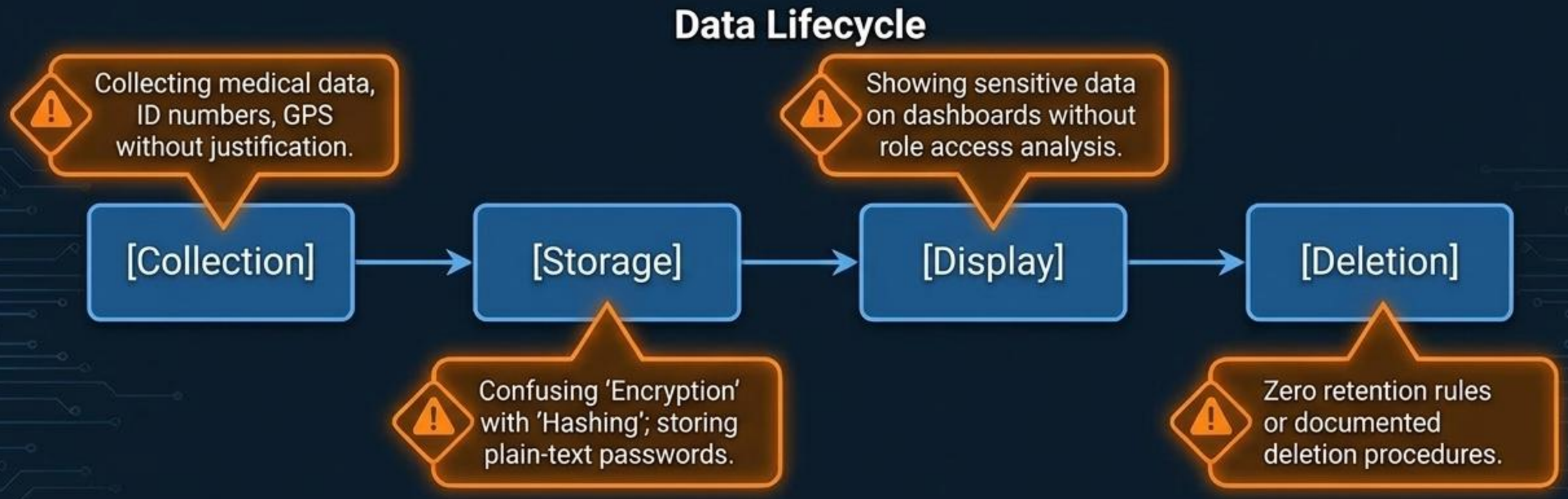
Critical Unaddressed Questions:

- What prevents two users from booking the same resource simultaneously?
- What happens if payment is recorded but the status update fails?
- How are failed transactions securely rolled back?

The Mandate: Identify at least two critical transactions in your MVP and formally document their integrity rules.

Gemini Notebook

Security is an Architecture, Not a Promise



The Reality

"The system will be secure" is a wish, not a design.

Functional login screens generated by AI do not equate to system security.

Required Stage 4 Artifacts

- Data classification matrix
- Role-permission mapping
- Security test cases
- Strict Authentication/Authorisation architecture

Gemini Notebook

Interface designs implied alignment rather than demonstrating it.

Finding:
GUI-to-requirement alignment was implied.

Evidence:
Only 10% of students provided explicit links between screens and requirements. New features or user roles appeared in the UI without documented changes in earlier phases.

Stage 4 Risk:
This is not just a design flaw; it is a failure of traceability and controlled evolution. Unlinked screens become un-testable code.

Solution:
Every UI element must map directly to an approved Phase 2 requirement and business rule.



Gemini Notebook

De-Mystifying the ‘AI Feature’

The Magic Box
(Cohort Submission)



>50% of projects named AI features but proposed fixed rules, keyword matching, or simple sorting.

The existence of an interface does not prove the existence of an AI component.

The AI Component Card
(Engineering Standard)



If you are claiming an AI feature, you must define the **data source, target variable, false positive/negative costs**, and a **manual non-AI fallback strategy.**

Gemini Notebook

Establishing Technical Ownership

Document Control

- Consistent system naming
- Visible current versions
- Formal change logs
- Structured repository

Project Planning

- Moving beyond generic Gantt phases
- Stage 4 Backlog created
- Linked to MVP requirements
- Scheduled integration time

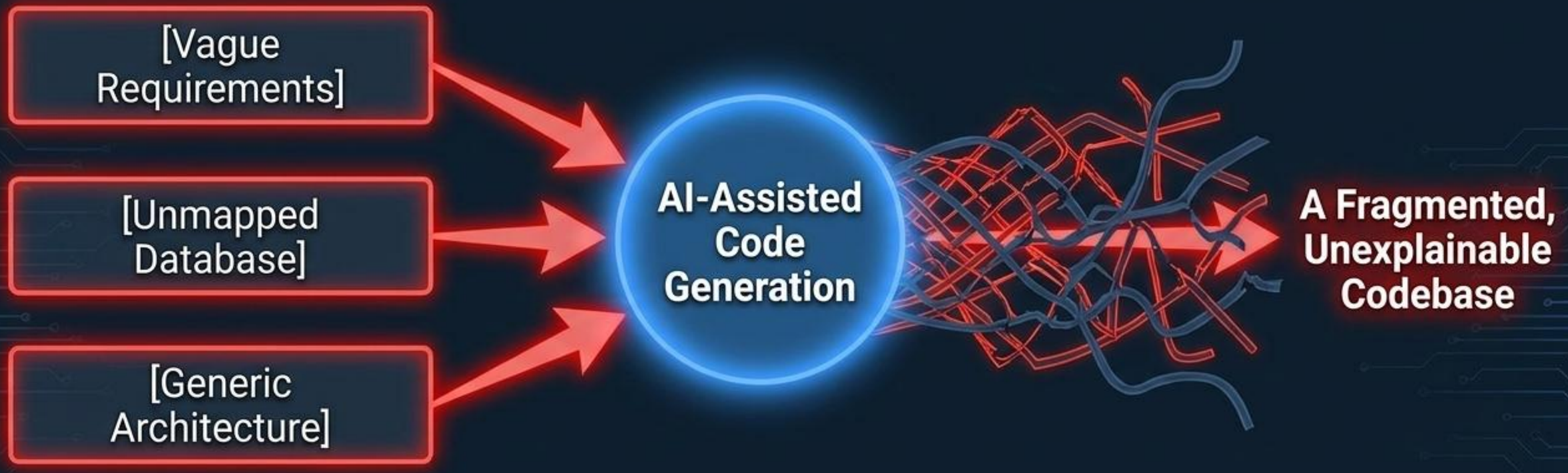
Artefact Command

- Ability to explain any requirement
- Ability to modify any diagram
- Ability to trace a feature to code
- Ability to explain AI decisions

Workforce readiness is not demonstrated by possessing a polished document. It is demonstrated by technical control over the artifacts. If you cannot control your documentation, you will fail to control code versions and database migrations.

Gemini Notebook

SYNTHESIS: The Stage 4 Risk of AI Generation



Why this matters now: AI tools will happily fill the gaps in your documentation with their own assumptions. If your baseline is weak, the AI will invent new frameworks and database rules. You will lose technical ownership of your own system, rendering it ungradable.

Gemini Notebook

The Self-Evaluation Diagnostic Matrix

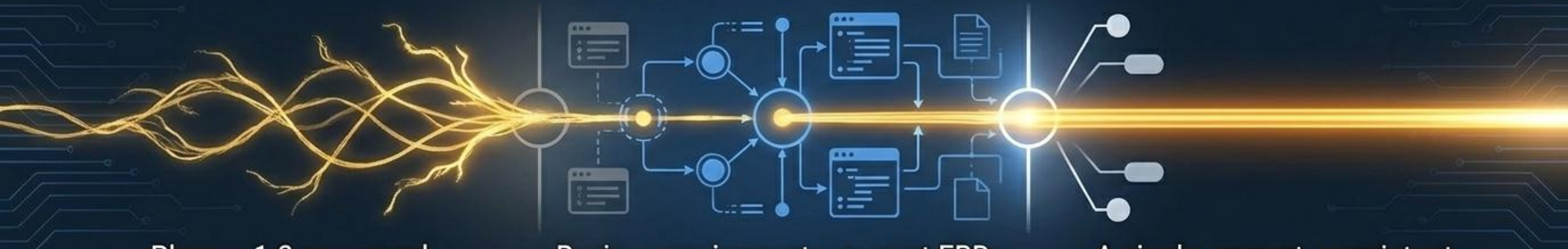
Diagnostic Question	Assessment
Scope & Requirements	
Is my MVP realistic and supported by evidence?	✓ [Yes] ~ [Partially] ✗ [No]
Does every requirement have a permanent ID and testable outcome?	✓ [Yes] ~ [Partially] ✗ [No]
Models	
Do my diagrams use consistent names and map to my architecture?	✓ [Yes] ~ [Partially] ✗ [No]
Do state diagrams model actual entity lifecycles?	✓ [Yes] ~ [Partially] ✗ [No]
Database	
Do my tables match my domain model?	✓ [Yes] ~ [Partially] ✗ [No]
Are correct cardinalities and constraints defined?	✓ [Yes] ~ [Partially] ✗ [No]
Interfaces	
Have I designed empty, error, and permission-denied states?	✓ [Yes] ~ [Partially] ✗ [No]
Does every screen support an approved requirement?	✓ [Yes] ~ [Partially] ✗ [No]

Audit your project against this matrix. Any 'No' identifies a critical technical risk that must be addressed before proceeding to Stage 4.

Gemini Notebook

The Consolidation Mandate: Securing the Baseline

Phase A	Phase B	Phase C
Original Submissions	Consolidation & Correction	Approved Stage 4 Baseline



Phases 1-3 preserved as academic record.

Review requirements, correct ERDs, link interfaces, maintain change log.

A single, correct, consistent, and traceable document.

The Objective:

Not to produce more pages, but to produce a traceable, testable baseline.

The Opportunity:

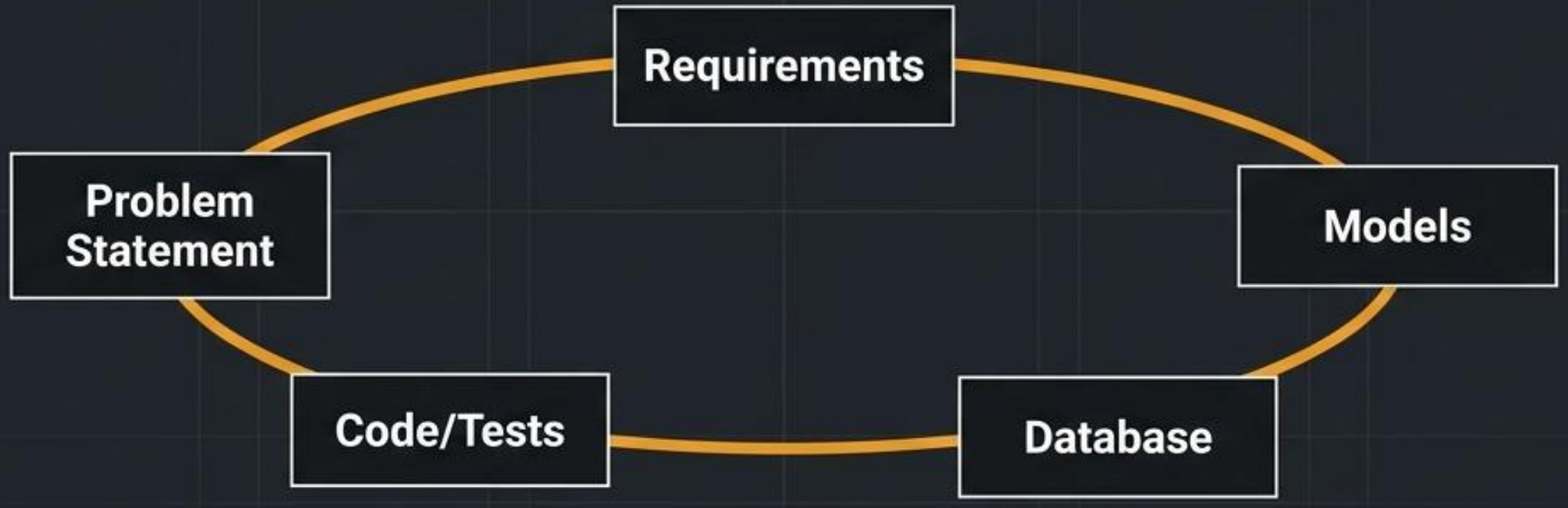
Reconcile your documents into one approved version before final grading and implementation.

The Standard:

The problem statement, requirements, models, database, and code must describe the same system.

Gemini Notebook

Engineering a Reliable Information System



Software engineering is not about producing pages; it is about producing proof.

By the end of Stage 4, your problem statement, requirements, models, database, code, and tests must describe the exact same system. Every important change must be deliberate, justified, and recorded. That is the standard expected of a graduate preparing to enter the software-development workforce.

Gemini Notebook